



VITYARTHI FLIPPED COURSE PROJECT

CampusConnect

College Event Management System

NAME: VIKRANT BARNWAL

REG. NO: 25BAI10558

COURSE: CSE3003 (PROGRAMMING IN JAVA)

SLOT: B14+D21

1. Introduction

CampusConnect is a Java-based command-line College Event Management System designed to centralize student, organizer, event, registration, attendance and reporting operations. The application uses Java collections for in-memory processing and text files for persistent storage.

1.1 Scope

The system covers the event lifecycle from organizer registration and event creation through student registration, participant viewing, attendance generation and reporting. It is implemented as a lightweight local application without an external database.

1.2 Target Users

- Students who browse and register for college events.
- Organizers who create and manage events and attendance.
- Faculty/project evaluators reviewing implementation, design and testing.

2. Problem Statement

College event information can become difficult to manage when student records, event details, registrations and attendance are maintained manually or in separate places. CampusConnect provides one console application to organize these activities, validate common inputs, reduce duplicate operations and maintain persistent records.

2.1 Objectives

- Centralize college event management.
- Maintain student and organizer records.
- Support event creation, modification and deletion.
- Support event registration and cancellation.
- Manage event capacity and duplicate-registration prevention.
- Generate attendance and participation reports.
- Demonstrate Java OOP, collections, File I/O, exception handling and validation.

3. Functional Requirements

Module	Functions
Student Management	Register student; browse events; register/cancel; view schedule and participation history.
Event Management	Create, update, delete and browse events; maintain event capacity.
Registration Management	Create/cancel registrations; validate entities; prevent duplicates; check capacity.
Attendance Management	Generate and store attendance; show attendance information and summaries.
Reporting	Dashboard, event registration, student participation and attendance summaries.

4. Non-Functional Requirements

Requirement	Description
Usability	Menu-driven CLI with clear prompts and numbered choices.
Reliability	Validation, duplicate checks and file persistence reduce invalid records and data loss during normal execution.
Maintainability	Responsibilities are separated into focused Java classes.
Error Handling	Invalid numeric input, empty fields, invalid emails and IDs are handled with user-facing messages.
Resource Efficiency	Uses lightweight collections and text files; no external database is required.
Scalability	Modular design permits later migration to database-backed or web implementations.
Security	Current version has no authentication; role-based login is a future enhancement.

5. Technical Requirements

Software: JDK 17 or later; Windows/Linux/macOS; Command Prompt, PowerShell or Terminal; Git for version control.

Development: Any Java-compatible IDE/editor is optional. **Hardware:** Dual-core processor, 4 GB RAM and approximately 100 MB available storage are sufficient for the lightweight CLI application.

No external Java libraries are required.

6. System Architecture

The logical architecture separates user interaction, domain objects, validation/persistence utilities and storage. Main coordinates the CLI workflow; Student, Event, Organizer, Registration and Attendance represent domain data; Validator and FileManager provide common services; Report produces summaries.

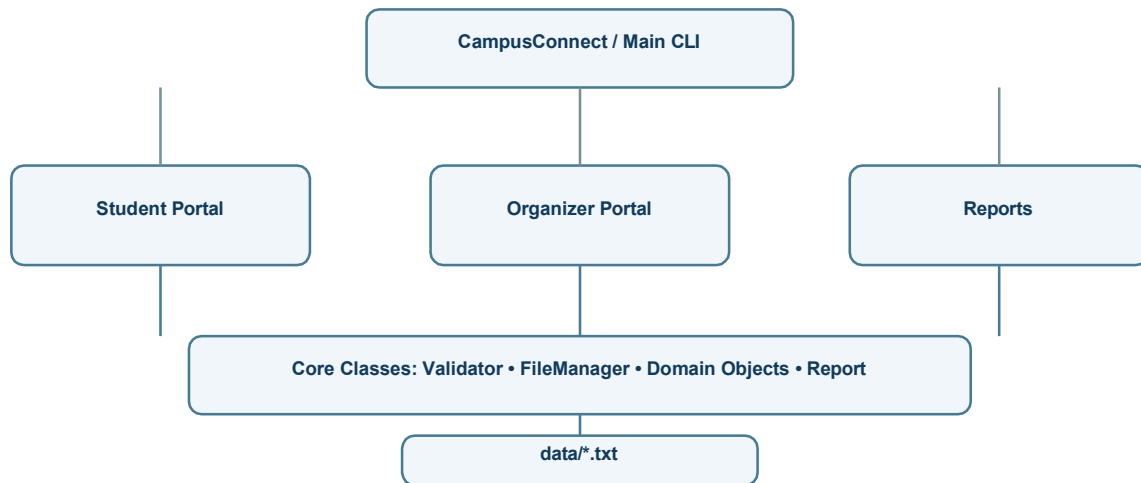


Figure 1. Logical system architecture.

7. Design Diagrams

7.1 Workflow Diagram

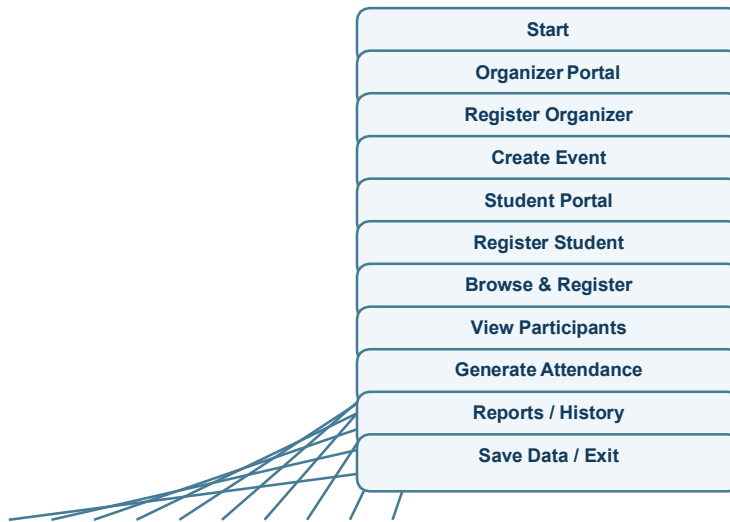


Figure 2. High-level demonstration workflow.

7.2 Use Case View

Actors: Student and Organizer. Major student use cases are registration, event browsing, event registration/cancellation and history. Major organizer use cases are organizer registration, event CRUD, participant viewing and attendance generation. Reports provide dashboard and summary outputs.

7.3 Class / Component Diagram

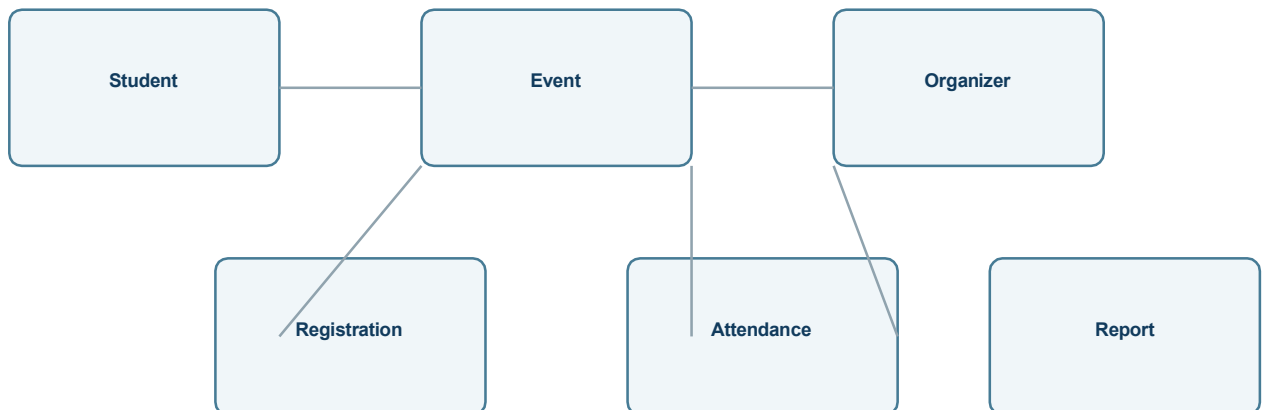


Figure 3. Simplified UML-style relationship view.

8. Sequence and Storage Design

8.1 Example Registration Sequence

Student selects the Student Portal → Main invokes registration workflow → Registration validates student/event and duplicate/capacity rules → FileManager persists the record → result is returned to the user.

8.2 Storage Design

File	Logical Records
students.txt	Student ID, student details and related information.
organizers.txt	Organizer ID and organizer details.
events.txt	Event ID, event details, capacity and organizer ownership.
registrations.txt	Student-event registration records.
attendance.txt	Student-event attendance records.

Records are pipe-separated and the data directory is created automatically when necessary. The project does not use a relational database; therefore the storage diagram is treated as a logical ER-style representation rather than a database schema.

8.3 Main Menu

Option	Operation
1	Student Portal
2	Organizer Portal
3	Reports
4	Save Data
5	Exit

9. Design Decisions & Rationale

Decision	Rationale
CLI interface	Keeps interaction simple and focuses the project on Java programming concepts.
ArrayList / Java Collections	Provides straightforward in-memory record management.
Text-file persistence	Avoids database setup while preserving records.
Separate classes	Improves modularity and demonstrates object-oriented design.
Central validation	Provides consistent input and constraint checking.
Ownership checks	Restrict event modification/deletion to the responsible organizer.
ValidationTest	Provides a repeatable way to test important validation/domain behaviours.

10. Implementation Details

Class	Responsibility
Main.java	Application entry point, main menu and portal navigation.
Student.java	Student information and student operations.
Organizer.java	Organizer information and organizer operations.
Event.java	Event information and event-management operations.
Registration.java	Student-event registration records.
Attendance.java	Attendance records.
Report.java	Dashboard and summary reports.
Validator.java	Input, email, empty-field and ID validation.
FileManager.java	Reading/writing persistent text files.
ValidationTest.java	Validation and related project checks.

11. Screenshots / Results

The following screenshots provide visual evidence of the implemented CampusConnect workflow, arranged in the same logical order as the demonstration flow: application launch, organizer and student registration, event creation and browsing, event registration, participation history, participant viewing, attendance generation, persistent storage and validation testing.

Note: the supplied screenshots include application execution, text-file persistence and automated validation evidence.

```
=====
                        CAMPUSCONNECT – EVENT MANAGEMENT
=====
1. Student Portal
2. Organizer Portal
3. Reports
4. Save Data
5. Exit
-----
Enter choice: |
```

Figure 4. CampusConnect main menu

```
-----
Enter choice: 2

===== ORGANIZER PORTAL =====
1. Register Organizer
2. Create Event
3. Update Event
4. Delete Event
5. View Participants
6. Generate Attendance
7. Back
Enter choice: 1
Enter organizer ID: 1
Enter name: Vikrant
Enter email: vikrant@example.com
Enter department/club: DSA club
Organizer registered successfully.
```

Figure 5. Organizer registration

```
===== ORGANIZER PORTAL =====  
1. Register Organizer  
2. Create Event  
3. Update Event  
4. Delete Event  
5. View Participants  
6. Generate Attendance  
7. Back  
Enter choice: 2  
Enter organizer ID: 1  
Event name: DSA got latent  
Date (DD-MM-YYYY): 12-09-2026  
Time (e.g. 10:30 AM): 12:30  
Venue: AB01  
Capacity: 500  
Event created successfully. Event ID: 101
```

Figure 6. Event creation

```
===== STUDENT PORTAL =====  
1. Register  
2. Browse Events  
3. Register for Event  
4. Cancel Registration  
5. View Schedule  
6. View Participation History  
7. Back  
Enter choice: 1  
Enter student ID: 22  
Enter full name: Vikrant Barnwal  
Enter email: student@example.com  
Enter course: Computer Science  
Student registered successfully.
```

Figure 7. Student registration

```

===== STUDENT PORTAL =====
1. Register
2. Browse Events
3. Register for Event
4. Cancel Registration
5. View Schedule
6. View Participation History
7. Back
Enter choice: 2

===== AVAILABLE EVENTS =====
ID: 101 | DSA got latent | 12-09-2026 | 12:30 | AB01
Capacity: 0/500
Organizer ID: 1
=====

```

Figure 8. Browsing available events

```

===== STUDENT PORTAL =====
1. Register
2. Browse Events
3. Register for Event
4. Cancel Registration
5. View Schedule
6. View Participation History
7. Back
Enter choice: 3
Enter student ID: 22

===== AVAILABLE EVENTS =====
ID: 101 | DSA got latent | 12-09-2026 | 12:30 | AB01
Capacity: 0/500
Organizer ID: 1
=====

Enter event ID: 101
Registration successful. Registration ID: 1001
===== STUDENT PORTAL =====

```

Figure 9. Student event registration

```

===== STUDENT PORTAL =====
1. Register
2. Browse Events
3. Register for Event
4. Cancel Registration
5. View Schedule
6. View Participation History
7. Back
Enter choice: 6
Enter student ID: 22

----- PARTICIPATION HISTORY -----
DSA got latent | 12-09-2026 12:30 | Attendance not marked
-----

```

Figure 10. Student participation history

```

=====
CAMPUSCONNECT - EVENT MANAGEMENT
=====
1. Student Portal
2. Organizer Portal
3. Reports
4. Save Data
5. Exit
-----
Enter choice: 2

===== ORGANIZER PORTAL =====
1. Register Organizer
2. Create Event
3. Update Event
4. Delete Event
5. View Participants
6. Generate Attendance
7. Back
Enter choice: 5
Enter organizer ID: 1
Event ID: 101

Participants for: DSA got latent
Registration ID: 1001 | Vikrant Barnwal | student@example.com | Computer Science

```

Figure 11. Organizer participant list

```
===== ORGANIZER PORTAL =====  
1. Register Organizer  
2. Create Event  
3. Update Event  
4. Delete Event  
5. View Participants  
6. Generate Attendance  
7. Back  
Enter choice: 6  
Enter organizer ID: 1  
Event ID: 101  
Is Vikrant Barnwal present? (Y/N): Y  
Attendance saved for 1 registered student(s).
```

Figure 12. Attendance generation

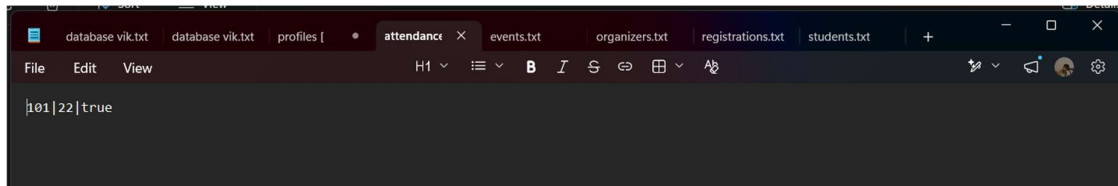


Figure 13. attendance.txt persistence record

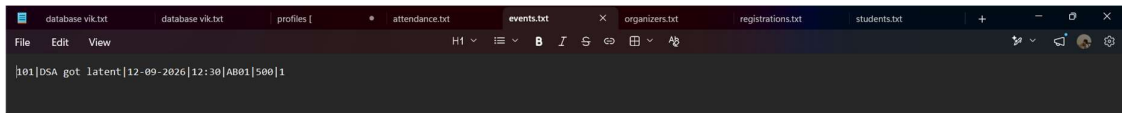


Figure 14. events.txt persistence record

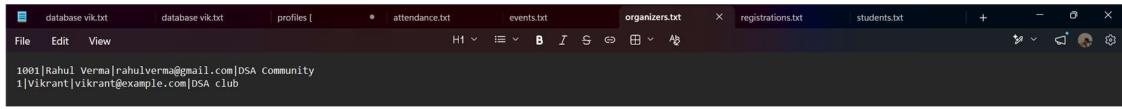


Figure 15. organizers.txt persistence records

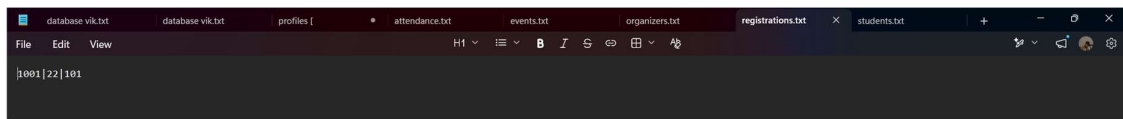


Figure 16. registrations.txt persistence record

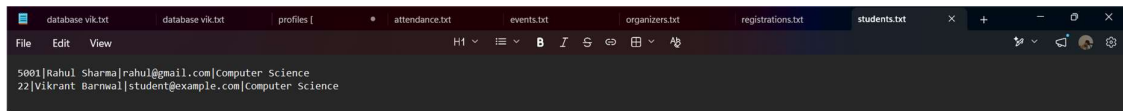


Figure 17. students.txt persistence records

```

C:\Users\Vikrant Barnwal\Downloads\vityarthi-project-java\vityarthi_project_java>javac *.java
C:\Users\Vikrant Barnwal\Downloads\vityarthi-project-java\vityarthi_project_java>java ValidationTest
===== CAMPUSCONNECT VALIDATION TEST =====
PASS : Valid email
PASS : Invalid email rejected
PASS : Valid name
PASS : Invalid name rejected
PASS : Student search
PASS : Event search
PASS : Registration lookup
PASS : Capacity counting
PASS : Attendance lookup
PASS : Present status
=====
Validation tests completed.
C:\Users\Vikrant Barnwal\Downloads\vityarthi-project-java\vityarthi_project_java>

```

Figure 18. Validation test results

12. Testing Approach

The project includes `ValidationTest.java`. Testing covers both valid and invalid cases, with emphasis on constraints that protect data integrity.

- Numeric input validation.
- Email validation.
- Empty-input validation.
- Duplicate student/organizer ID checks.
- Duplicate event-registration prevention.
- Event-capacity checking.
- Organizer ownership checking.
- File persistence checks.

Test commands: `javac *.java` followed by `java ValidationTest`. The project documentation specifies PASS results for validation, student, event, registration and attendance checks.

13. Challenges Faced

- Keeping in-memory collections and text-file persistence consistent.
- Preventing duplicate registrations while enforcing event capacity.
- Checking organizer ownership for event modification/deletion.
- Handling invalid console input without terminating normal execution.
- Maintaining modularity across multiple related workflows without an external framework or database.

14. Learnings & Key Takeaways

- Applied object-oriented programming using multiple domain classes.
- Used Java Collections Framework, particularly ArrayList.
- Implemented File I/O for persistence.
- Practised exception handling and input validation.
- Designed a multi-module CLI application and documented it using diagrams.
- Used Git/GitHub for version control.
- Understood how the system can evolve into a database-backed or web application.

15. Future Enhancements

Enhancement	Description
JDBC / MySQL	Replace text files with structured database persistence.
Authentication	Add login and role-based access for students and organizers.
QR attendance	Support faster event check-in.
Notifications	Send event reminders, confirmations and updates.
GUI / Web interface	Provide graphical or browser-based access.
Search & filtering	Filter events by date, venue or category.
CSV / PDF export	Export reports for administration and sharing.
Enhanced analytics	Add richer dashboards and participation trends.

16. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines. The supplied guideline defines the project objective, functional and non-functional requirements, technical expectations, required design artefacts, GitHub contents and PDF report sections.
2. Java standard platform APIs and language concepts used by the implementation: collections, File I/O, exceptions and object-oriented programming.
3. Git and GitHub used for version control and repository hosting.

17. VITyarthi Report Compliance Checklist

Required Report Section	Included
1. Cover Page	Yes
2. Introduction	Yes
3. Problem Statement	Yes
4. Functional Requirements	Yes
5. Non-functional Requirements	Yes
6. System Architecture	Yes
7. Use Case / Workflow / Sequence / Class / Storage Design	Yes
8. Design Decisions & Rationale	Yes
9. Implementation Details	Yes
10. Screenshots / Results	Actual application screenshots included
11. Testing Approach	Yes
12. Challenges Faced	Yes
13. Learnings & Key Takeaways	Yes
14. Future Enhancements	Yes
15. References	Yes

End of Report